



TEST DRIVEN DEVELOPMENT — CHERRY-TECH

**Daniel Alejandro Sánchez Ahumada | 23310411 | Gestión de
Servicios TI**

¿QUÉ ES TDD?

Test Driven Development (TDD) es una metodología de desarrollo de software en la que las pruebas se escriben antes que el código. El ciclo se repite en tres pasos conocidos como **Red-Green-Refactor**:

Red: Escribir una prueba que falle porque la funcionalidad aún no existe. **Green:** Escribir el código mínimo necesario para que esa prueba pase. **Refactor:** Mejorar el código sin romper la prueba ya aprobada.

Este enfoque invierte el proceso tradicional. En lugar de desarrollar y luego probar, el desarrollador primero define qué debe hacer el sistema (la prueba), y luego lo construye para cumplir esa definición. Esto garantiza que cada línea de código producida tiene un propósito verificable y que el sistema nunca avanza sin estar respaldado por evidencia de funcionamiento.

La base teórica del TDD fue formalizada por Kent Beck en el marco del Extreme Programming (XP). Su principio central es que el código de prueba y el código de producción son igualmente importantes: ambos se mantienen, versionan y mejoran juntos.

HERRAMIENTAS DE TDD SELECCIONADAS PARA CHERRY-TECH

Dado que Cherry-Tech utiliza una arquitectura de microservicios con Python en el backend (MS-VPD-Engine, MS-Automation, MS-Ingestor, MS-History-Analytics) y JavaScript/React en el frontend (MS-User-Interface), se seleccionaron las siguientes herramientas:

Pytest — Framework de pruebas para Python. Es el estándar de la industria para proyectos Python por su sintaxis simple, soporte de fixtures, parametrización de pruebas y plugins como pytest-cov para medir cobertura de código. Se usa en todos los microservicios del backend.

pytest-cov — Plugin de Pytest que genera reportes de cobertura de código. Permite verificar que el pipeline CI/CD bloquee cualquier merge con cobertura menor al 80% definido en el SLA del proyecto.

pytest-mock — Plugin para crear objetos mock (simulaciones) de dependencias externas como el broker MQTT o InfluxDB, permitiendo probar la lógica de negocio de forma aislada sin necesidad de tener los servicios externos activos.

Jest — Framework de pruebas para JavaScript. Se usa en el MS-User-Interface para probar los componentes React del dashboard, la lógica de alertas y la comunicación con la API REST.

Postman / Newman — Para pruebas de integración de los endpoints REST entre microservicios. Newman permite ejecutar las colecciones de Postman desde el pipeline CI/CD de forma automatizada.

TDD APLICADO A LAS HISTORIAS DE USUARIO

A continuación se desarrolla el ciclo TDD completo para las historias de usuario más críticas del proyecto, siguiendo el orden Red → Green → Refactor.

HISTORIA 9 — Motor de Cálculo VPD (MS-VPD-Engine)

Historia: Como sistema (MS-VPD-Engine), quiero calcular el VPD automáticamente con la fórmula estándar de la ASABE cada vez que recibo datos de los sensores.

Criterios de Aceptación: Precisión de 2 decimales. Retornar error

`InvalidDataException` si los sensores envían valores fuera de rango físico (humedad < 0% o > 100%).

FASE RED

```
# tests/test_vpd_engine.py
import pytest
from src.vpd_engine import calcular_vpd, InvalidDataException

class TestCalcularVPD:

    def test_calculo_correcto_condiciones_optimas(self):
        """
        Dado que la temperatura es 22°C y la humedad relativa es 65%,
        el VPD calculado debe ser 0.97 kPa con precisión de 2 decimales.
        Rango óptimo para tomate Cherry según ASABE: 0.8 - 1.2 kPa
        """
        resultado = calcular_vpd(temperatura=22.0, humedad=65.0)
        assert resultado == 0.97

    def test_calculo_vpd_temperatura_alta(self):
        """
        Dado que la temperatura sube a 28°C con humedad del 70%,
        el VPD debe calcularse correctamente y superar el umbral.
        """
        resultado = calcular_vpd(temperatura=28.0, humedad=70.0)
        assert resultado == 1.14

    def test_precision_dos_decimales(self):
        """
        El resultado debe tener exactamente 2 decimales.
```

```

"""
resultado = calcular_vpd(temperatura=25.0, humedad=60.0)
assert resultado == round(resultado, 2)

```

```

def test_humedad_cero_lanza_excepcion(self):
    """
    Si la humedad es 0%, los datos son físicamente imposibles.
    Debe lanzarse InvalidDataException para proteger los actuadores.
    """
    with pytest.raises(InvalidDataException):
        calcular_vpd(temperatura=22.0, humedad=0.0)

```

```

def test_humedad_mayor_100_lanza_excepcion(self):
    """
    Una humedad superior al 100% indica sensor dañado.
    Debe lanzarse InvalidDataException.
    """
    with pytest.raises(InvalidDataException):
        calcular_vpd(temperatura=22.0, humedad=105.0)

```

```

def test_temperatura_negativa_lanza_excepcion(self):
    """
    Temperatura negativa en un invernadero es físicamente imposible.
    Debe rechazarse para evitar cálculos erróneos.
    """
    with pytest.raises(InvalidDataException):
        calcular_vpd(temperatura=-5.0, humedad=65.0)

```

```

def test_temperatura_extrema_lanza_excepcion(self):
    """
    Una temperatura de 80°C indica sensor con falla crítica.
    """
    with pytest.raises(InvalidDataException):
        calcular_vpd(temperatura=80.0, humedad=65.0)

```

En este punto, al ejecutar `pytest`, todas las pruebas fallan porque `vpd_engine.py` no existe. Eso es correcto: estamos en fase RED.

FASE GREEN

```

# src/vpd_engine.py
import math

```

```

class InvalidDataException(Exception):
    """Se lanza cuando los datos del sensor están fuera del rango físico válido."""
    pass

```

```

def calcular_vpd(temperatura: float, humedad: float) -> float:
    """
    Calcula el Vapor Pressure Deficit (VPD) usando la fórmula estándar ASABE.
    """

```

Fórmula:

$SVP = 0.6108 * e^{(17.27 * T / (T + 237.3))}$ [kPa] - Presión de vapor saturado
 $AVP = SVP * (HR / 100)$ [kPa] - Presión de vapor actual
 $VPD = SVP - AVP$ [kPa] - Déficit de presión de vapor

Args:

temperatura: Temperatura del aire en °C (rango válido: 0 - 50°C)
humedad: Humedad relativa en % (rango válido: 1 - 100%)

Returns:

VPD en kPa redondeado a 2 decimales.

Raises:

InvalidDataException: Si los parámetros están fuera del rango físico válido.

"""

Validación de rangos físicos

if not (1.0 <= humedad <= 100.0):

raise InvalidDataException(

f"Humedad fuera de rango: {humedad}%. Rango válido: 1% - 100%"

)

if not (0.0 <= temperatura <= 50.0):

raise InvalidDataException(

f"Temperatura fuera de rango: {temperatura}°C. Rango válido: 0°C - 50°C"

)

Cálculo de la presión de vapor saturado (SVP)

svp = 0.6108 * math.exp((17.27 * temperatura) / (temperatura + 237.3))

Presión de vapor actual (AVP)

avp = svp * (humedad / 100.0)

VPD

vpd = svp - avp

return round(vpd, 2)

Ejecutar `pytest` ahora: todas las pruebas pasan. Estamos en fase GREEN.

FASE REFACTOR

src/vpd_engine.py (refactorizado)

import math

from dataclasses import dataclass

TEMP_MIN, TEMP_MAX = 0.0, 50.0

HUMEDAD_MIN, HUMEDAD_MAX = 1.0, 100.0

@dataclass

class VPDResult:

vpd: float

```

    svp: float
    avp: float
    estado: str # "optimo", "bajo", "alto"

def _validar_parametros(temperatura: float, humedad: float) -> None:
    if not (HUMEDAD_MIN <= humedad <= HUMEDAD_MAX):
        raise InvalidDataException(f"Humedad fuera de rango: {humedad}%")
    if not (TEMP_MIN <= temperatura <= TEMP_MAX):
        raise InvalidDataException(f"Temperatura fuera de rango: {temperatura}°C")

def _calcular_svp(temperatura: float) -> float:
    return 0.6108 * math.exp((17.27 * temperatura) / (temperatura + 237.3))

def calcular_vpd(temperatura: float, humedad: float) -> float:
    _validar_parametros(temperatura, humedad)
    svp = _calcular_svp(temperatura)
    avp = svp * (humedad / 100.0)
    return round(svp - avp, 2)

def calcular_vpd_detallado(temperatura: float, humedad: float) -> VPDResult:
    """Versión extendida que retorna contexto completo para el dashboard."""
    _validar_parametros(temperatura, humedad)
    svp = _calcular_svp(temperatura)
    avp = svp * (humedad / 100.0)
    vpd = round(svp - avp, 2)
    estado = "optimo" if 0.8 <= vpd <= 1.2 else ("bajo" if vpd < 0.8 else "alto")
    return VPDResult(vpd=vpd, svp=round(svp, 4), avp=round(avp, 4), estado=estado)

```

HISTORIA 10 — Modo de Supervivencia (MS-Automation)

Historia: Como sistema (MS-Automation), quiero ejecutar un ciclo de riego local ante la pérdida de conexión con la nube.

Criterios de Aceptación: Activar **LOCAL_SURVIVAL_MODE** tras 30 segundos sin latido de red. El riego debe seguir el horario guardado en caché.

FASE RED

```

# tests/test_automation.py
import pytest
from unittest.mock import MagicMock, patch
from src.automation import AutomationController, SurvivalMode

class TestModoSupervivencia:

```

```
def setup_method(self):
    self.cache_horario = {
        "riego_inicio": "06:00",
        "riego_duracion_min": 20,
        "ventilacion_umbral_vpd": 1.2
    }
    self.controller = AutomationController(cache=self.cache_horario)
```

```
def test_modos_supervivencia_se_activa_tras_30_segundos(self):
    """
    Tras 30s sin recibir latido de red, el modo de supervivencia
    debe activarse automáticamente.
    """
    self.controller.registrar_perdida_conexion(segundos_sin_red=30)
    assert self.controller.modos_activo == SurvivalMode.LOCAL
```

```
def test_modos_supervivencia_no_activa_antes_de_30_segundos(self):
    """
    Con menos de 30s sin red, el sistema debe mantener
    el modo normal sin cambios.
    """
    self.controller.registrar_perdida_conexion(segundos_sin_red=15)
    assert self.controller.modos_activo == SurvivalMode.CLOUD
```

```
def test_riego_usa_horario_de_cache(self):
    """
    En modo supervivencia, el riego debe ejecutarse según
    el horario guardado en caché, no esperar instrucciones de la nube.
    """
    self.controller.registrar_perdida_conexion(segundos_sin_red=30)
    horario = self.controller.obtener_horario_riego()
    assert horario["inicio"] == "06:00"
    assert horario["duracion_min"] == 20
```

```
def test_modos_cloud_se_restaura_al_recuperar_conexion(self):
    """
    Al recuperar la conexión, el sistema debe volver al modo nube
    y sincronizar los datos del buffer local.
    """
    self.controller.registrar_perdida_conexion(segundos_sin_red=30)
    self.controller.registrar_recuperacion_conexion()
    assert self.controller.modos_activo == SurvivalMode.CLOUD
```

```
def test_buffer_local_registra_eventos_durante_desconexion(self):
    """
    Todos los eventos (riego, alertas) durante la desconexión
    deben guardarse en el buffer para sincronizarse al volver la red.
    """
```

```
self.controller.registrar_perdida_conexion(segundos_sin_red=30)
self.controller.ejecutar_riego_local()
assert len(self.controller.buffer_eventos) > 0
```

FASE GREEN

```
# src/automation.py
from enum import Enum
from datetime import datetime
```

```
class SurvivalMode(Enum):
    CLOUD = "cloud"
    LOCAL = "local"
```

```
UMBRAL_SUPERVIVENCIA_SEG = 30
```

```
class AutomationController:
```

```
    def __init__(self, cache: dict):
        self.cache = cache
        self.modos_activo = SurvivalMode.CLOUD
        self.buffer_eventos = []
```

```
    def registrar_perdida_conexion(self, segundos_sin_red: int):
        if segundos_sin_red >= UMBRAL_SUPERVIVENCIA_SEG:
            self.modos_activo = SurvivalMode.LOCAL
```

```
    def registrar_recuperacion_conexion(self):
        self.modos_activo = SurvivalMode.CLOUD
```

```
    def obtener_horario_riego(self) -> dict:
        return {
            "inicio": self.cache.get("riego_inicio"),
            "duracion_min": self.cache.get("riego_duracion_min")
        }
```

```
    def ejecutar_riego_local(self):
        evento = {
            "tipo": "riego",
            "modo": self.modos_activo.value,
            "timestamp": datetime.now().isoformat(),
            "horario": self.obtener_horario_riego()
        }
        self.buffer_eventos.append(evento)
        return evento
```


HISTORIA 11 — Filtrado de Outliers (MS-History-Analytics)

Historia: Como sistema (MS-History-Analytics), quiero descartar automáticamente lecturas fuera de rango para evitar falsas activaciones de actuadores.

Criterios de Aceptación: Si un valor se desvía más de 3 sigmas respecto al promedio, ignorarlo. Loguear el evento de sensor sospechoso.

FASE RED

```
# tests/test_analytics.py
import pytest
from src.analytics import filtrar_outliers, SensorSospechoso

class TestFiltrarOutliers:

    def test_lectura_normal_no_es_descartada(self):
        """
        Lecturas dentro del rango estadístico normal deben pasar el filtro.
        """
        historial = [22.1, 22.3, 22.0, 22.4, 22.2, 21.9, 22.1]
        nueva_lectura = 22.2
        resultado = filtrar_outliers(nueva_lectura, historial)
        assert resultado.es_valida == True

    def test_lectura_80_grados_es_outlier(self):
        """
        Un sensor que reporta 80°C en un invernadero de tomate cherry
        (promedio ~22°C) es claramente un error de hardware.
        Debe ser descartado para no activar ventiladores incorrectamente.
        """
        historial = [22.1, 22.3, 22.0, 22.4, 22.2, 21.9, 22.1]
        lectura_erronea = 80.0
        resultado = filtrar_outliers(lectura_erronea, historial)
        assert resultado.es_valida == False

    def test_outlier_genera_log_de_sensor_sospechoso(self):
        """
        Un outlier debe generar un registro de alerta sobre el sensor.
        """
        historial = [22.1, 22.3, 22.0, 22.4, 22.2, 21.9, 22.1]
        resultado = filtrar_outliers(80.0, historial)
        assert resultado.alerta is not None
        assert isinstance(resultado.alerta, SensorSospechoso)
```

```
def test_historial_insuficiente_acepta_lectura(self):
    """
```

Con menos de 5 datos históricos no hay suficiente contexto estadístico. La lectura se acepta por defecto.

"""

```
historial = [22.0, 22.1]
```

```
resultado = filtrar_outliers(22.5, historial)
```

```
assert resultado.es_valida == True
```

```
def test_desviacion_exacta_3_sigma_es_aceptada(self):
```

"""

El umbral es estrictamente mayor a 3 sigmas. En el límite exacto, la lectura se acepta.

"""

```
import statistics
```

```
historial = [22.0] * 10
```

```
sigma = statistics.stdev(historial + [22.0])
```

```
# Con datos uniformes sigma es 0, ajustamos para la prueba
```

```
historial = [20.0, 22.0, 24.0, 21.0, 23.0, 22.0, 21.5, 22.5]
```

```
media = statistics.mean(historial)
```

```
sigma = statistics.stdev(historial)
```

```
en_limite = media + (3 * sigma)
```

```
resultado = filtrar_outliers(round(en_limite, 2), historial)
```

```
assert resultado.es_valida == True
```

FASE GREEN

```
# src/analytics.py
```

```
import statistics
```

```
from dataclasses import dataclass
```

```
from typing import Optional, List
```

```
from datetime import datetime
```

```
MINIMO_HISTORIAL = 5
```

```
UMBRAL_SIGMA = 3.0
```

```
@dataclass
```

```
class SensorSospechoso:
```

```
    valor_detectado: float
```

```
    media_historial: float
```

```
    desviacion: float
```

```
    timestamp: str
```

```
@dataclass
```

```
class ResultadoFiltro:
```

```
    es_valida: bool
```

```
    valor: float
```

```
    alerta: Optional[SensorSospechoso] = None
```

```
def filtrar_outliers(lectura: float, historial: List[float]) -> ResultadoFiltro:
```

```

"""
Aplica el criterio de 3-sigma para identificar lecturas anómalas.
"""
if len(historial) < MINIMO_HISTORIAL:
    return ResultadoFiltro(es_valida=True, valor=lectura)

media = statistics.mean(historial)
sigma = statistics.stdev(historial)

if sigma == 0:
    return ResultadoFiltro(es_valida=True, valor=lectura)

distancia = abs(lectura - media)

if distancia > UMBRAL_SIGMA * sigma:
    alerta = SensorSospechoso(
        valor_detectado=lectura,
        media_historial=round(media, 2),
        desviacion=round(sigma, 4),
        timestamp=datetime.now().isoformat()
    )
    return ResultadoFiltro(es_valida=False, valor=lectura, alerta=alerta)

return ResultadoFiltro(es_valida=True, valor=lectura)

```

HISTORIA 1 — Visualización de VPD en Dashboard (MS-User-Interface)

Historia: Como agrónomo, quiero visualizar el valor actual del VPD en un panel central para determinar si las plantas requieren intervención.

Criterios de Aceptación: Actualización cada 5 segundos. El sistema debe marcar en rojo si el VPD está fuera del rango 0.8 - 1.2 kPa.

FASE RED

```

// tests/vpd_display.test.js
import { getVPDStatus, formatVPD } from '../src/utils/vpd_display';

describe('VPD Display Logic', () => {

  test('VPD dentro del rango óptimo retorna estado "optimo"', () => {
    expect(getVPDStatus(0.95)).toBe('optimo');
    expect(getVPDStatus(0.80)).toBe('optimo');
    expect(getVPDStatus(1.20)).toBe('optimo');
  });
});

```

```
test('VPD por debajo de 0.8 retorna estado "bajo" y color rojo', () => {  
  const resultado = getVPDStatus(0.6);  
  expect(resultado).toBe('bajo');  
});
```

```
test('VPD por encima de 1.2 retorna estado "alto" y color rojo', () => {  
  const resultado = getVPDStatus(1.5);  
  expect(resultado).toBe('alto');  
});
```

```
test('VPD formateado muestra exactamente 2 decimales', () => {  
  expect(formatVPD(0.9)).toBe('0.90 kPa');  
  expect(formatVPD(1.234)).toBe('1.23 kPa');  
});
```

```
test('VPD nulo o indefinido retorna estado "sin datos"', () => {  
  expect(getVPDStatus(null)).toBe('sin_datos');  
  expect(getVPDStatus(undefined)).toBe('sin_datos');  
});
```

```
});
```

FASE GREEN

```
// src/utils/vpd_display.js
```

```
const VPD_MIN = 0.8;  
const VPD_MAX = 1.2;
```

```
export function getVPDStatus(vpd) {  
  if (vpd === null || vpd === undefined) return 'sin_datos';  
  if (vpd < VPD_MIN) return 'bajo';  
  if (vpd > VPD_MAX) return 'alto';  
  return 'optimo';  
}
```

```
export function formatVPD(vpd) {  
  return `${parseFloat(vpd).toFixed(2)} kPa`;  
}
```

HISTORIA 15 — Circuit Breaker (MS-Automation)

Historia: Como sistema, quiero implementar un Circuit Breaker para aislar fallas en servicios externos.

Criterios de Aceptación: Abrir circuito tras 3 fallos consecutivos. Intentar reconexión tras 5 minutos en estado "abierto".

FASE RED

```
# tests/test_circuit_breaker.py
import pytest
import time
from src.circuit_breaker import CircuitBreaker, CircuitState

class TestCircuitBreaker:

    def setup_method(self):
        self.cb = CircuitBreaker(max_fallos=3, timeout_segundos=300)

    def test_estado_inicial_es_cerrado(self):
        assert self.cb.estado == CircuitState.CERRADO

    def test_circuito_abre_tras_3_fallos_consecutivos(self):
        for _ in range(3):
            self.cb.registrar_fallo()
        assert self.cb.estado == CircuitState.ABIERTO

    def test_circuito_no_abre_con_2_fallos(self):
        for _ in range(2):
            self.cb.registrar_fallo()
        assert self.cb.estado == CircuitState.CERRADO

    def test_exito_reinicia_contador_de_fallos(self):
        self.cb.registrar_fallo()
        self.cb.registrar_fallo()
        self.cb.registrar_exito()
        self.cb.registrar_fallo()
        assert self.cb.estado == CircuitState.CERRADO

    def test_circuito_permite_llamada_en_estado_semiabierto(self):
        """
        Tras el timeout, el circuito pasa a SEMI_ABIERTO
        y permite una llamada de prueba.
        """
        cb_rapido = CircuitBreaker(max_fallos=3, timeout_segundos=0)
        for _ in range(3):
            cb_rapido.registrar_fallo()
```

```
assert cb_rapido.puede_intentar() == True
```

```
def test_circuito_abierto_bloquea_llamadas(self):  
    for _ in range(3):  
        self.cb.registrar_fallo()  
    assert self.cb.puede_intentar() == False
```

FASE GREEN

```
# src/circuit_breaker.py  
from enum import Enum  
from datetime import datetime, timedelta
```

```
class CircuitState(Enum):  
    CERRADO = "cerrado"  
    ABIERTO = "abierto"  
    SEMI_ABIERTO = "semi_abierto"
```

```
class CircuitBreaker:
```

```
    def __init__(self, max_fallos: int = 3, timeout_segundos: int = 300):  
        self.max_fallos = max_fallos  
        self.timeout = timedelta(seconds=timeout_segundos)  
        self.fallos_consecutivos = 0  
        self.estado = CircuitState.CERRADO  
        self.tiempo_apertura = None
```

```
    def registrar_fallo(self):  
        self.fallos_consecutivos += 1  
        if self.fallos_consecutivos >= self.max_fallos:  
            self.estado = CircuitState.ABIERTO  
            self.tiempo_apertura = datetime.now()
```

```
    def registrar_exito(self):  
        self.fallos_consecutivos = 0  
        self.estado = CircuitState.CERRADO  
        self.tiempo_apertura = None
```

```
    def puede_intentar(self) -> bool:  
        if self.estado == CircuitState.CERRADO:  
            return True  
        if self.estado == CircuitState.ABIERTO:  
            if datetime.now() - self.tiempo_apertura >= self.timeout:  
                self.estado = CircuitState.SEMI_ABIERTO  
                return True  
            return False
```

```
return True # SEMI_ABIERTO permite una llamada de prueba
```

ESTRUCTURA DEL PROYECTO CON TDD

```
cherry-tech/
├── src/
│   ├── vpd_engine.py    # Lógica de cálculo VPD
│   ├── automation.py    # Control de actuadores y modo supervivencia
│   ├── analytics.py     # Filtrado de outliers y análisis histórico
│   ├── circuit_breaker.py # Patrón Circuit Breaker
│   └── utils/
│       └── vpd_display.js # Lógica de presentación del dashboard
├── tests/
│   ├── test_vpd_engine.py # 7 pruebas — Historia 9
│   ├── test_automation.py # 5 pruebas — Historia 10
│   ├── test_analytics.py  # 5 pruebas — Historia 11
│   ├── test_circuit_breaker.py # 6 pruebas — Historia 15
│   └── vpd_display.test.js # 5 pruebas — Historia 1
├── pytest.ini           # Configuración de cobertura mínima
├── .github/
│   └── workflows/
│       └── cherry-ci.yml # Pipeline que ejecuta las pruebas automáticamente
```

```
# pytest.ini
[pytest]
addopts = --cov=src --cov-fail-under=80 --cov-report=term-missing
testpaths = tests
```

RESUMEN DE COBERTURA POR HISTORIA

Historia	Módulo	Pruebas escritas	Cobertura objetivo
H-9 Motor VPD	vpd_engine.py	7 pruebas	95%
H-10 Modo Supervivencia	automation.py	5 pruebas	95%
H-11 Filtrado Outliers	analytics.py	5 pruebas	90%
H-1 Dashboard VPD	vpd_display.js	5 pruebas	85%

H-15 Circuit Breaker	circuit_breaker.py	6 pruebas	90%
Total	5 módulos	28 pruebas	> 80% global

CONCLUSIÓN

TDD invierte la forma de pensar del desarrollador: en lugar de construir y luego verificar si funciona, primero defines con precisión qué debe hacer el sistema y construyes exclusivamente lo necesario para cumplirlo. Aplicado a Cherry-Tech, esto fue especialmente valioso en el módulo VPD-Engine porque obligó a definir los límites físicos válidos de los sensores antes de escribir una sola línea de lógica de cálculo, lo que habría sido difícil de detectar en un proceso de desarrollo tradicional.